Anteater Chess
Version 1.0

EECS 22L: Software Engineering Project

University of California, Irvine

Team Name:
Hikaru Naka-more

Team Members:
Jonghyun Choi: 39487186
Vinh Nguyen: 76073226
Hamza Faisal: 29338975
Richard Nguyen: 30864762
William Kuang: 58552955
Kun Liang: 61607118

Date: April 7, 2026

Table of Contents

- Glossary of Implementation Terms
- 1. Software Architecture Overview
    - 1.1 Main Data Types and Structures
    - 1.2 Major Software Components
    - 1.3 Module Interfaces
    - 1.4 Overall Program Flow
- 2. Installation
    - 2.1 System Requirements and Compatibility
    - 2.2 Setup and Configuration
    - 2.3 Building, Compilation, and Installation
- 3. Documentation of Packages, Modules, and Interfaces
    - 3.1 Data Structures Descriptions
    - 3.2 Functions and Parameters Descriptions
    - 3.3 Input / Output Format Descriptions
- 4. Development Plan and Timeline
    - 4.1 Task Partitions
    - 4.2 Team Member Responsibilities
- Copyright / References / Index

Glossary of Implementation Terms

- **Piece**: A chess object with a colour and type
- **Board**: The 8x10 Anteater Chess board that stores all current piece positions.
- **Piece Type**: King, Queen, Rook, Bishop, Knight, Anteater, or Pawn.
- **Move**: Go from one square to another, featuring unique moves like promotion, ant eating, en passant, and casting.
- **Legal Move**: A move allowed by the game rules that does not leave the moving side in check.
- **Checkmate**: Occurs when a king is in check and cannot be removed by any legal action.
- **Check**: A state in which the king is being attacked.
- **Ant Eating**: Nearby pawns may be eaten in a single move during a unique anteater capture sequence.

1. Software Architecture Overview

1.1 Main Data Types and Structures

White and black players are distinguished by a color type. To determine whether a piece is a king, queen, rook, bishop, knight, anteater, or pawn, utilise the PieceType type. In addition to combining color and type, a piece also keeps track of its previous movements, which is essential for applying special rules like castling and a pawn's first two-step move.

1.2 Major Software Components

GUI - Chess players will have a Graphical User Interface in which they can visualize the board, both players' pieces in corresponding positions, with the player's turns. This interface will be developed from C, utilizing X11 as our display protocol, allowing us to render graphics client-side.

Board Representation - A data structure to hold all important information of the chess software, piece positions, game state, special checks, and overall performance of the game.

Move Gen/Validator - Generates and checks legal moves for all pieces at a given time, adhering to all established chess rules (check, castling, en-passant, mate, etc).

Engine - Chess engine with stored move list to determine the next best move based on the difficulty.

```
+-------------------------------------------------------+
|                        main.c                         |
| (Game Loop, State Management, Terminal UI, Player I/O) |
+-------------------------------------------------------+
           |                                |
           v                                v
+----------------------+         +------------------------+
|     MoveList.c       |         |   MoveValidation.c     |
| (Legal Move Generator| ------> | (Rules Engine, Check/  |
|  for future AI)      |         |  Mate Detection)       |
+----------------------+         +------------------------+
           |                                |
           |                                |
           v                                v
+-------------------------------------------------------+
|                      GameData.c                       |
|  (Core Data Structures, Board State, ASCII Rendering,  |
|   Raw Memory Manipulation)                            |
+-------------------------------------------------------+
```

[Figure 1 - Module Hierarchy Diagram]

1.3 Module Interfaces

The major software components of the chess system, as listed above, will be able to communicate through defined module interfaces that can perform its assigned tasks while remaining logically separate. Modular interaction will assist in testing, expansion, and maintenance of the software.

GUI ⇔ Board Representation - The GUI retrieves board and game state information from the board representation/data structure, and then is able to display the visual positions, whilst updating placement, turn info, etc. The GUI (depending on implementation) can also send player input back to the board representation to forward the game state.

Board Representation ⇔ Move Gen/Validator - The move generator and validator both rely on the board representation as well to be able to utilize current piece locations and information, as well as the game state. They need this data to both check and confirm validity, making sure board representation is legal and can be updated accordingly. These interactions will ensure that all moves made will be legal and no faults outside of classical chess rules will happen.

Move Generator/Validator ⇔ Engine - The engine will also utilize the move generator and validator to determine legal moves before eventually evaluating the “most optimal” set of possible outcomes/moves. This will assist in the engine being able to only make and search for valid decisions, narrowing down the best move possibilities, and improving efficiency.

Engine ⇔ Board Representation - The engine will read and use the current board/game state in order to evaluate the current position, future best positions, and compare move selections. Once a move is made, the result will be passed to the board to forward the game state.

Engine ⇔ GUI - In matches involving the chess bot, the engine will have to provide the chosen move to the system, and the GUI needs to be able to reflect that visually on its display to the player. This will allow for both sides to see the chess game state representation concurrently.

1.4 Overall Program Flow

In the main() function of the Anteater Chess program, a Board data structure is instantiated and initialised through a call to InitializeBoard(). This function clears the board and arranges every piece in the 8x10 grid in its initial locations. Once the application has been initialised, it prints a welcome message and asks the user to choose a side by typing "w" for white or "b" for black. To guarantee that only valid selections are allowed, this input is validated in a loop. Any errors or incorrect inputs are securely removed from the input buffer. In accordance with traditional chess rules, the program initialises the turn system with white moving first and allocates the opposite colour to the AI after a valid option is made.

The program will then begin its main execution loop, which keeps going until a termination condition is satisfied. PrintBoard() displays the board as a formatted ASCII grid with rows labelled numerically and columns labelled alphabetically from A through J at the beginning of each iteration, displaying the board's current state in the terminal. The currentTurn variable is then used by the program to determine whose turn it is. The user is asked to enter a move in the format 'ColumnRow ColumnRow' (for instance, 'E2 E4') if it is the player's turn. To prevent buffer overflow problems, the input is received as a complete line using fgets(). The source and destination coordinates are then extracted using sscanf(). In order to access the board structure, these coordinates are transformed from their character-based representation into zero-based array indices.

Bounds checking is done by the computer before the move is applied to make sure the source and destination positions are inside the board's valid dimensions. MovePiece() is used to update the board by copying the piece to the destination square and clearing the old location; it is used to carry out the move if the input is properly formatted and within boundaries. If the input is invalid due to out-of-range coordinates or improper formatting, the application shows an error notice and prompts the user again without advancing the game state.

2. Installation

2.1 System Requirements and Compatibility

This chess software is developed completely in the C programming language and uses the X11 display system for the graphical user interface. Therefore, the software is intended primarily for Unix-like environments that can support the X Window system.

2.1.1 Minimum System Requirements

Processor: 64-bit x86 processor equivalent

Memory: 512 MB RAM minimum

Storage: 50MB of available disk space

Display: X11-Compliant

Compiler: GCC or another compatible C compiler

Build Tools: make utility

Libraries: X11 development, mathlib, stdout

2.1.2 Supported Operating Systems

Software is expected to run on Linux distributions with X11 support, Unix-based machines, WSL/Linux environments with X11 configuration

2.1.3 Compatibility X11

Additional compatibility layers may be necessary depending on the machine.

Windows: WSL on X server, or Linux virtual machine with X11 forwarding

macOS: XQuartz or another X11-compatible environment

2.2 Setup and Configuration

2.2.1 Required Packages

Before compilation and running, the following tools and libraries must be installed:

- A C compiler such as GCC
- Make a build utility
- X11 runtime and development libraries

2.2.2 Environment Configuration

Before building the software, the user should ensure that:

- The compiler will be accessible from the command line
- X11 libraries are installed and forwarded
- Project files are in their intended working directory
- All required libraries and environment variables are properly set

For remote or SSH sessions, X11 forwarding may need to be enabled. You can use the command

echo $DISPLAY - to show a valid value that indicates graphical output can be directed on the server.

2.2.3 Configuration Options

Depending on the implementation, configuration may include:

- AI difficulty
- Default game mode
- Algebraic move notation window

2.3 Building, Compilation, and Installation

2.3.1 Obtaining Source Code

First, the code must be copied/cloned into a working directory:

Example:

```
git clone <repo_url>
cd chess_project
```

If the project is distributed as a folder, the user will need to extract it and navigate accordingly.

2.3.2 Building the Program

Compilation will be handled through a makefile that will compile all required source files and link them into the final executable file. The exact command build rule will depend on the project file organization and hierarchy.

Compiler flags used will -Wall, -g to enable common warnings and for debugging purposes.

X11 will have to be handled as well for rendering and event handling in the GUI, information regarding this as stated above in earlier sections.

2.3.3 Installation Process

After following the compilation and build rules, the program can be run directly using the ./(program_name) command. A system-wide installation may become a necessary add-on for mass client usage.

To remove compiled files and rebuild from scratch, a clean rule will be provided in the make file, such as:

```
make clean
```

This will target object files, temp files, and previous executables to make sure outdated files will not interfere with future builds.

2.3.4 Post-Installation

After building and running, the user should verify successful installation by launching the application and confirming the following:

- The program opens with an X11 window successfully
- The chessboard is rendered completely
- Input is detected, and pieces can be moved and selected

- All classical game rules are enforced
- AI responds when chosen

A successful launch confirms all of these rules and verifies that the installation was completed successfully.

3. Documentation of Packages, Modules, and Interfaces

3.1 Data Structure Descriptions

Piece
The Piece structure has two characters for color and type. It is to help make all the squares of the board easier to see and print. Each square that is blank will have blank character values.

```
typedef struct {
    char color;
    char type;
} Piece;
```

[Figure 2 - Piece Definition]

Board

The Board structure has a grid of two-dimensional pieces arrayed in a grid. This is an authoritative representation of the current match. All move validation and rendering logic are based on this grid for reading and updating.

```
typedef struct {
    Piece grid[ROWS][COLS];
} Board;
```

[Figure 3 - Board Definition]

Move

The Move structure has four integers: sourceRow, sourceCol, targetRow, and targetCol. The Move structure is intended to standardise how a move is represented when it is saved or passed between modules.

```
typedef struct {
    int fRow;
    int fCol;
    int tRow;
    int tCol;
} Move;
```

[Figure 4 - Move Definition]

MoveList

The MoveList structure will hold up to a maximum of 1024 candidate legal moves, as well as a counter of how many moves are currently stored. This will make the move generator reusable for AI search logic later. Note in Figure 5 that the macro MAX_MOVES is defined to a value of 1024.

```
typedef struct {
    Move moves[MAX_MOVES];
    int count;
} MoveList;
```

[Figure 5 - MoveList Definition]

3.2 Functions and Parameter Descriptions

InitializeBoard

- Description - Sets the initial configuration of the 8x10 Anteater Chess board, clearing all squares and placing pieces in their starting positions.
- Parameter(s) - Board* pBoard - A pointer to the Board structure that will be modified in place to represent the starting state.

PrintBoard

- Description - Renders the current game state to the terminal as a formatted ASCII grid.
- Parameter(s) - Board* pBoard - A Pointer to the Board structure containing the current piece positions and game data to be displayed.

MovePiece

- Description - A low-level function that executes the translation of a piece from one coordinate to another without performing any rule validation.
- Parameter(s) :
    - Board* pBoard - A pointer to the active game board.
    - int oRank - The original rank (row) of the piece.
    - int oFile - The original file (column) of the piece.
    - int nRank - The new destination rank of the piece.
    - int nFile - The new destination file of the piece.

AnteaterCapture

- Description - Executes the specialized multi-capture sequence for the Anteater piece, clearing all enemy pawns along a straight path and relocating the Anteater to the final destination.
- Parameter(s) :
    - Board* pBoard - A pointer to the active game board.
    - int oRank/oFile - The original coordinates of the Anteater.
    - int nRank/nFile - The destination coordinates (the last pawn eaten).
    - char color - The color (w or b) of the Anteater performing the capture.

IsValidMove

- Description - The primary entry point for the rules engine. It validates moves against piece-specific rules and ensures the moving player's King is not left in a state of check.
- Parameter(s) :

- Board* pBoard - Pointer to the board state against which the move is being validated.
- int fRow/fCol - The "from" coordinates (source).
- int tRow/tCol - The "to" coordinates (destination).
- char color - The color of the player attempting the move.

DetermineAIMove

- Description - The entry point for the computer player's strategy module. It evaluates the board and returns an optimal move based on the selected difficulty.
- Parameters:
  - Board* pBoard - Pointer to the current authoritative board representation.
  - char aiColor - The color assigned to the AI player ('w' or 'b').
  - char difficulty - A character representing the AI's search depth and evaluation logic ('1' for Easy, '2' for Medium, '3' for Hard).

3.3 Input / Output Format Descriptions

<u>User Input Format</u>

User input for piece movement is captured via the standard input stream using a buffered fgets read to prevent buffer overflow vulnerabilities. The input string is parsed using sscanf to extract the source and destination coordinates.

- Syntax - [SourceFile][SourceRank] [DestinationFile][DestinationRank]
- Format Constraints - File range A through J (case-insensitive) and Rank range 1 through 8
- Example: “E2 E4” to move a piece at File E, Rank 2 to File E, Rank 4
  - Anteater Capture - “D6 G7” to move an anteater at D6 to G7, automatically validating and clearing all ants in the horizontal or vertical path

Standard Output Format

The game state is rendered to the standard output as an ASCII-formatted grid as an 8 x 10 matrix.

- Grid Structure - Ranks are labelled numerically from 8 down to 1 on the Y-axis. Files are labelled alphabetically from A to J on the X-axis.
- Piece Notation - Each occupied tile prints a two-character sequence enclosed in spaces
    - Character 1 (color): ‘w’ for White or ‘b’ for Black
    - Character 2 (Type): ‘K’ for King, ‘Q’ for Queen, ‘R’ for Rook, ‘B’ for Bishop, ‘N’ for Knight, ‘A’ for Anteater, and ‘P’ for Pawn / Ant
    - Empty spaces are rendered as four blank spaces

File Output Format (Game Log)

The software will sequentially log every executed move to a persistent text file (e.g. game_log.txt).

- Syntax - [Turn #]. [Color]: [Source] -> [Destination] [Optional Flags]
- Log Structure Rules: Moves are recorded only after passing a certain function. Standard movies log the coordinate translation, while more complex moves are appended with specific flag types for easy parsing by external analysis tools.

```
1. White: E2 -> E4
1. Black: E7 -> E5
2. White: G1 -> F3
2. Black: D7 -> D6
3. White: D1 -> G4 (Anteater Capture)
3. Black: A7 -> A6
4. White: E4 -> D5 (En Passant)
```

[Figure 6 - Movement Log File Example]

4. Development Plan and Timeline

4.1 Task Partitions

Core Logic & Rules *(Hamza & Kun)* **-** Implement chess rules, including the unique Anteater "ant eating" moves and legal move validation.

Chess Bot *(Jay & Vinh)* - Development of the strategy module for the computer player.

User Interfaces *(Vinh & William)* - Manage both the initial textual (ASCII) interface and eventual transition to GUI implementation.

Data Management *(Richard)* - Design the chess objects (ex. pieces, board, and moves) and the log file module.

Quality Assurance *(All Members)* - Lead the testing and debugging efforts to ensure no illegal moves occur during the user's gameplay experience.

4.2 Development Timeline

Phase	Date Started & Target Date	Key Deliverables & Activities
Chess User Manual	April 3 - April 6, 2026	Brainstormed foundational implementation ideas during this phase. Submitted "Chess_UserManual.pdf" to UCI servers on April 5.
Alpha Development	April 3 - April 20, 2026	Foundation code started by Vinh and Richard for general program flow via terminal for UI and beginner UX. Merged William's updated terminal flow code with Vinh's to enhance UX. Merged Hamza's starting code for move validation to further UX. Merged joint effort of Kun's and Jay's enhancement of move validation in main program flow and track move lists at a current moment. Chess bot implementation researched

		and done by joint effort of all team members.
Chess Software Document	April 7 - April 13, 2026	Brought over ideas from Chess User Manual and fine tuned to be more detailed. Submitted “Chess_SoftwareSpec.pdf” and updated “Chess_UserManual.pdf” to UCI servers on April 12.
Alpha Release	April 3 - April 20, 2026	Add some more code if the team sees fit and continue testing and debugging. Submit “Chess_Alpha.tar.gz” to UCI servers by April 20.
Software Release	April 3 - April 27, 2026	Add final stages of development if necessary to address potential bugs. Submit “Chess_V1.0.tar.gz” to UCI servers by April 27.

Back Matter

Copyright

©2026 Team Hikaru Naka-more, University of California, Irvine.

Index

Anteater Capture, 11, 12

ASCII Interface, 11

Board Structure, 9

Checkmate, 3

GUI (Graphical User Interface), 3, 4, 12

Make/Makefile, 6, 8

MoveList, 10